

Selective Forgetting: A Graph-Based Memory Framework for Long-Term LLM Agents

Theo Rusu

Department of Computer Science
Toronto Metropolitan University
Toronto, Ontario, Canada
trusu@torontomu.ca

Sourena Khanzadeh*

The Creative School
Toronto Metropolitan University
Toronto, Ontario, Canada
Flybits, Creative AI Hub
Toronto, Ontario, Canada
sourena.khanzadeh@torontomu.ca

Manar Alalfi

Department of Computer Science
Toronto Metropolitan University
Toronto, Ontario, Canada
manar.alalfi@torontomu.ca

Abstract

Knowledge graphs have been proposed as a structured alternative to flat retrieval-augmented generation for long-term agent memory, on the assumption that representing conversations as entities and relations improves recall. We evaluate that assumption directly. Our framework extracts each conversational turn into typed nodes and attributed edges, answers questions from a two-hop subgraph, and periodically prunes nodes that score low on a weighted combination of recency, access frequency, degree centrality, and age. On LongMemEval, the graph does not outperform a flat vector baseline at a matched candidate-generation budget of five retrieval roots: token F1 is 0.417 against 0.468, and a paired bootstrap over 500 questions gives $\Delta = -0.050$ (95% CI $[-0.085, -0.016]$). The gap is widest on questions that require recalling a specific prior assistant turn, where judged correctness falls from 0.911 to 0.607, suggesting that decomposing a turn into entities discards the surface form these questions depend on. The forgetting module is more successful. Applied once to a persistent 27,021-node graph, it removes 9.8% of nodes and 9.5% of stored bytes; token F1 is unchanged ($+0.001$, 95% CI $[-0.015, +0.016]$) and judged correctness falls by 1.6 points, with the 95% interval bounding any loss at 3.8 points ($[-0.038, +0.006]$). Because our extractor is a single small model evaluated on one benchmark, these results characterise this extraction-based pipeline rather than graph-structured memory in general.

Code: <https://github.com/skhanzad/Selective-Amnesia>

1 Introduction

Large Language Models have rapidly evolved from standalone text generators into the foundation of complex, agentic systems that are able to reason, plan, and use external tools. These systems are increasingly being adopted as personal AI assistants that interact with users over prolonged periods of time. The quality of these interactions is reliant not only on the model’s immediate

*This work was conducted prior to the author’s affiliations with Flybits and The Creative School. The author was previously affiliated with the Department of Computer Science at Toronto Metropolitan University.

reasoning ability, but also on the model’s capacity to incorporate information from past exchanges. As such, memory becomes a critical component of these systems and is typically classified into two subcategories: short-term memory and long-term memory.

The current widely adopted long-term memory approach is retrieval-augmented generation Lewis et al. [2020], where dense vector stores are indexed and queried at inference time to retrieve relevant entries that augment the model’s output. This method models memory as a flat, similarity-based retrieval system, which has been shown to be sensitive to noise, prone to retrieving irrelevant or redundant context, and is limited in its ability to support multi-hop reasoning or maintain coherent long-term knowledge Gao et al. [2023].

To address these limitations, recent work has explored structured memory representations based on knowledge graphs, where information is organized as entities and their relationships rather than independent embeddings. In these systems, memories are encoded as nodes and edges, allowing more complex semantic and relational representations to be stored Ji et al. [2021], Peng et al. [2023].

However, existing graph-based approaches mainly focus on how information is added and maintained for consistency Chhikara et al. [2025]. This does not address a fundamental challenge of long-term memory systems: unbounded growth. As interactions accumulate over time, memory stores become increasingly large, creating a range of downstream negative effects, including degraded retrieval quality, higher computational cost, and the retention of low-utility information.

As memory accumulates over prolonged interactions, the system must integrate new information and manage the relevance of existing knowledge. Prior work in continual learning and neural memory systems has shown that effective memory requires mechanisms for selective retention and forgetting, as retaining all information leads to performance degradation Kirkpatrick et al. [2017], Wei et al. [2026].

In this work, we investigate whether structuring long-term conversational memory as a knowledge graph meaningfully improves retrieval and reasoning in LLM agents, and whether systems can remain efficient over extended interactions. We introduce a graph-based memory framework with an explicit forgetting module that controls the lifecycle of stored information. Rather than assuming structural representations are uniformly beneficial, our study empirically characterizes where graph-based memory helps and where it degrades performance. In addition, we show that selective forgetting based on recency, frequency, and structural importance can reduce memory size without materially affecting retrieval quality. These findings highlight that effective long-term memory requires structured representation and careful design of update and retention mechanisms.

2 Related Work

2.1 Memory in LLM-based Agents

Equipping neural systems with explicit memory predates the current generation of language models. Early differentiable architectures such as Neural Turing Machines Graves et al. [2014] and End-to-End Memory Networks Sukhbaatar et al. [2015] coupled a controller with an addressable external store, establishing the read/write abstraction that later memory systems inherit. As LLMs became the backbone of agentic systems, memory was repurposed to persist information between turns and sessions rather than within a single forward pass Zhang et al. [2025]. A common design is the memory stream of Generative Agents Park et al. [2023], which logs observations and retrieves them using a combination of recency, importance, and relevance, and periodically synthesizes higher-level reflections. Subsequent systems extend this idea along different axes: MemoryBank Zhong et al. [2024] introduces an updating scheme inspired by the Ebbinghaus forgetting curve Ebbinghaus [1913]; MemGPT Packer et al. [2023] treats memory as an operating-system-style hierarchy that pages information between a bounded context window and external storage; ReadAgent Lee et al. [2024] compresses very long contexts into gist memories; and Think-in-Memory Liu et al. [2023], Self-Controlled Memory Wang et al. [2025], and MemLLM Modarressi et al. [2024] give the model explicit control over what is stored and recalled. These approaches also enable long-term dialogue settings studied by Xu et al. [2022]. Most of this line of work, however, organizes memory as a flat collection of entries and emphasizes writing and reading rather than principled removal.

2.2 Retrieval-Augmented Generation

The dominant strategy for grounding LLM outputs in external knowledge is retrieval-augmented generation Lewis et al. [2020], which retrieves relevant passages from a non-parametric store and conditions generation on them. Dense retrieval Karpukhin et al. [2020] and jointly pre-trained retrieval-reading models such as REALM Guu et al. [2020] and RETRO Borgeaud et al. [2022] improved retrieval quality on scale, while retrieval has been shown to reduce hallucinations in dialogue Shuster et al. [2021]. More recent variants add self-reflective control over when and what to retrieve Asai et al. [2024] and specialize models for conversational settings Liu et al. [2024]. However, as surveyed by Gao et al. [2023], RAG fundamentally models memory as a flat, similarity-based lookup over independent embeddings. This makes it sensitive to retrieval noise and redundancy and limits its capacity for multi-hop reasoning or maintaining coherent long-term knowledge, motivating more structured representations of memory.

2.3 Graph-Structured Memory

Knowledge graphs offer a structured alternative in which information is represented as entities and the relations between them Ji et al. [2021], Peng et al. [2023]. A growing body of work integrates such structure with LLMs Pan et al. [2024], ranging from prompting with retrieved triples Baek et al. [2023] to letting the model reason by traversing a graph Sun et al. [2024]. For retrieval specifically, GraphRAG Edge et al. [2024] constructs an entity graph and community summaries to support query-focused summarization, and HippoRAG Gutiérrez et al. [2024] draws on hippocampal indexing theory to combine a knowledge graph with graph-based retrieval for long-term recall. In the agent-memory setting, systems such as Mem0 Chhikara et al. [2025] adopt graph representations to store and consolidate user information across sessions. These methods demonstrate the benefits of relational structure for retrieval and reasoning, but they concentrate on how information is added and kept consistent and largely leave unbounded growth of the memory store unaddressed.

2.4 Forgetting and Memory Retention

The need to forget is well established outside of agent memory. In human cognition, retention decays predictably over time Ebbinghaus [1913]. In neural networks, naive sequential learning induces catastrophic forgetting McCloskey and Cohen [1989], prompting mechanisms that protect important parameters Kirkpatrick et al. [2017]; the broader phenomenon of forgetting in deep learning is surveyed by Wang et al. [2024], and machine unlearning studies the deliberate removal of specific information Bourtole et al. [2020]. A consistent finding across these areas is that effective memory requires selective retention rather than indefinite accumulation. This principle has only recently been applied to agent memory: FadeMem Wei et al. [2026] introduces biologically inspired forgetting to keep agent memory efficient. Our work is closest in spirit to this direction, but couples forgetting with a graph-structured store: rather than treating retention as a post-hoc filter over flat entries, we integrate a forgetting module into the life cycle of nodes and edges, so that obsolete or low-utility memories are removed while relational structure is preserved.

2.5 Evaluating Long-Term Memory

Assessing memory over extended interactions requires dedicated benchmarks. LoCoMo Maharana et al. [2024] evaluates very long-term conversational memory, and LongMemEval Wu et al. [2024] probes chat assistants on long-term interactive memory abilities such as multi-session reasoning and knowledge updates. We adopt LongMemEval to evaluate whether structured memory with forgetting sustains high-quality recall as interactions accumulate.

3 Methodology

This study proposes a graph-based conversational memory framework that models interactions as a structured, evolving knowledge graph. Rather than storing past exchanges as independent embeddings, the system maintains entities and their relationships as nodes and edges, continuously updates this graph as new conversational turns arrive, and periodically prunes low-importance nodes to bound graph growth over long interactions.

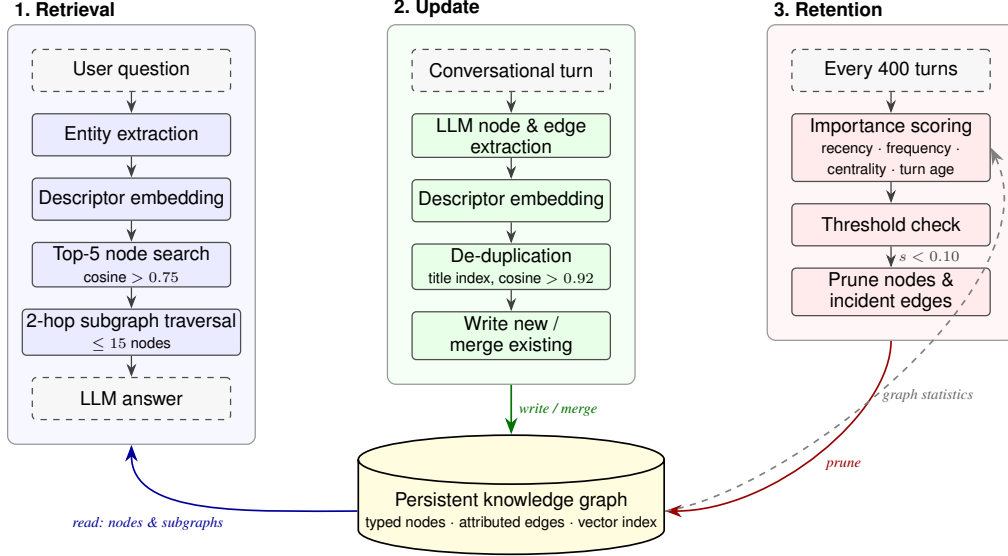


Figure 1: Overview of the proposed memory framework. Each conversational turn is processed by the *update* stage, which extracts, embeds, de-duplicates, and writes nodes and edges into the persistent knowledge graph. At question time, the *retrieval* stage embeds the referenced entities, selects the top-5 matching nodes ranked by cosine similarity, and expands them via a 2-hop subgraph traversal to build the answer-generation context. Every 400 turns, the *retention* stage scores each node by recency, access frequency, centrality, and turn age, pruning nodes whose importance falls below the threshold together with their incident edges.

3.1 Architecture

The framework is organized as a three-stage pipeline: *retrieval*, *update*, and *retention*. In the *retrieval* stage, the subgraph most relevant to the current question is selected and serialized as context for answer generation. In the *update* stage, an LLM extracts entities and relationships from the current conversational turn and integrates them into the knowledge graph. In the *retention* stage, a forgetting module scores every stored node and removes those whose importance falls below a threshold, bounding graph growth and discarding low-utility information. Figure 1 gives an overview of the full pipeline and how the three stages interact with the persistent knowledge graph.

3.2 Knowledge Graph Schema

The graph consists of typed nodes and edges with both types and attributes. Each node carries a label, a short title, a natural-language content description, a flat `attributes` dictionary, and system fields for temporal, access, and retention tracking (`created_at`, `access_count`, `last_accessed_at`, `turns_at_creation`, `importance_score`).

Nine node labels are defined in a fixed ontology: *Person*, *Organization*, *Location*, *Event*, *Concept*, *Artifact*, *Preference*, *Goal*, and *Skill*. Edges connect pairs of nodes via a typed relationship predicate and may carry their own `attributes` dictionary to capture relational properties such as duration, confidence, or quantity without introducing additional nodes.

3.3 Extraction Pipeline

Each conversational turn is processed by a single LLM extraction call (GPT-4o-mini) using a structured system prompt that defines the ontology, output schema, and extraction rules. Turns are prefixed with their speaker role (`[Role: user]` or `[Role: assistant]`) so the extractor handles each appropriately.

User and assistant turns are processed differently during extraction. For user turns, the system captures facts about the user, including their preferences, goals, skills, and relationships. Assistant

turns are processed in two modes: (a) user-related facts that the assistant references or confirms, and (b) factual claims, recommendations, and named-entity information stated by the assistant, enabling the system to recall information provided in prior turns.

Turns that contain only superfluous or generic filler words produce empty output.

The extractor returns strict JSON conforming to the graph schema. A validation layer rejects any output that fails to parse, references undefined ontology types, or contains structurally invalid node-edge references.

3.4 Embedding and De-duplication

Node descriptors are constructed from each node’s label, title, and content and embedded using `omic-embed-text` served locally via Ollama. The resulting vectors are stored alongside each node in the graph’s `vector_index` dictionary and persisted with the graph JSON file.

Before a new node is written to the graph, two de-duplication checks are applied in sequence. The first is title-based: if an existing node shares the same label and normalized title as the incoming node, it is identified as a match and the embedding check is skipped entirely. To enable the title-based check, the system maintains a `title_index` - a dictionary mapping each node’s normalized title to its node ID — which is updated whenever a new node is created, allowing $O(1)$ lookup without any vector computation. The second is embedding-based, applied only when the title check finds no match: the incoming node’s vector is compared to all stored vectors via linear cosine scan; if the nearest existing node has a cosine similarity above 0.92, it is treated as the same entity. When either check identifies a match, the matched node’s `access_count` and `last_accessed_at` fields are updated, providing a frequency signal to the importance scoring module before formal retrieval occurs.

3.5 Subgraph Retrieval

At inference time, a lightweight extraction call identifies the entities referenced by the current question and embeds their descriptors. An exhaustive cosine-similarity search is performed over all stored node embeddings. Nodes with a similarity score above 0.75 are retained and ranked by score. We use breadth-first search (BFS) starting from the top-5 retrieved nodes, expanding the graph up to two hops from these root nodes and stopping once a maximum of 15 nodes has been collected. Justifications for these parameter choices are provided in Appendix A.2.

The retrieved nodes and edges are serialized as a compact textual representation and included into the answer-generation prompt as contextual memory.

Visited nodes in a retrieved subgraph has its `access_count` incremented and `last_accessed_at` updated to the current session timestamp.

3.6 Importance Scoring and Forgetting

Each node is assigned an importance score that combines four components:

$$\text{Score} = w_r \cdot \text{recency}(t) + w_f \cdot \text{frequency}(c) \\ + w_c \cdot \text{centrality}(d) + w_t \cdot \text{turns_decay}(k),$$

where *recency* is an exponential decay from the node’s last access timestamp with a 90-day half-life; *frequency* is the log-normalized access count; *centrality* is the log-scaled edge degree; and *turns_decay* is an exponential decay from the node’s creation turn with a 1,000-turn half-life. The weights are set to $w_r = 0.35$, $w_f = 0.25$, $w_c = 0.20$, $w_t = 0.20$.

The forgetting module is invoked every 400 conversational turns. All nodes whose importance score falls below 0.10 are pruned, together with every edge incident to those nodes. Additional details are provided in Appendix A.2.

4 Experiments

This section presents two experiments designed to evaluate (1) whether a knowledge graph retrieval mechanism improves answer accuracy over a flat vector baseline, and (2) whether the proposed forgetting module can reduce graph storage without degrading retrieval quality. In all experiments, the underlying language model is held constant and only the retrieval and memory mechanisms are varied.

4.1 Dataset

Both experiments use the LongMemEval benchmark Wu et al. [2024], a dataset designed to evaluate long-term conversational memory in dialogue systems. The benchmark consists of 500 questions, each paired with a multi-session conversation history, referred to as the *haystack*, which contains the information required to answer the question. Haystack sessions span real historical dates covering approximately 33 months (June 2021 – February 2024), with each question drawing on between one and several sessions.

Questions are categorized into six types that probe distinct memory demands. *Single-session (user)* questions ask about facts the user stated directly, such as personal attributes or past events. *Single-session (assistant)* questions require recalling specific information the assistant provided in a prior turn, such as a recommendation or a factual explanation. *Single-session (preference)* questions target implicit or explicit user preferences expressed in conversation. *Knowledge-update* questions test whether the system correctly tracks values that changed across sessions, favoring the most recent statement over earlier ones. *Multi-session* questions require aggregating information spread across two or more separate sessions. *Temporal-reasoning* questions demand ordering events or computing time intervals from information embedded in the haystack.

4.2 Experiment 1: Retrieval Quality and Knowledge Retention

The goal of Experiment 1 is to determine whether structuring conversational memory as a knowledge graph improves retrieval quality over a flat vector baseline. For each of the 500 questions, a fresh knowledge graph is built from that question’s haystack sessions alone, then used to answer the question. The baseline stores the same haystack turns as raw text chunks in a flat vector store and retrieves the top-5 most similar chunks at query time. Both systems use the same language model for answer generation, and neither has access to information outside the question’s own haystack.

Performance is measured using token-level precision and F1-score, and an LLM-as-judge correctness score (binary, averaged across questions).

Table 1: Experiment 1 results: Graph RAG vs. Baseline RAG on LongMemEval. J = LLM-judge; Single = single-session.

Type	Graph RAG (Ours)				Baseline RAG			
	n	P	F1	J	n	P	F1	J
Single (user)	70	0.743	0.737	0.771	70	0.823	0.819	0.929
Single (asst.)	56	0.680	0.575	0.607	56	0.916	0.774	0.911
Single (pref.)	30	0.312	0.083	0.233	30	0.274	0.114	0.367
Know.-update	78	0.507	0.456	0.513	78	0.553	0.511	0.590
Multi-session	133	0.384	0.326	0.398	133	0.381	0.342	0.436
Temporal	133	0.468	0.328	0.293	133	0.416	0.334	0.278
Overall	500	0.505	0.417	0.454	500	0.532	0.468	0.536

4.3 Experiment 2: Long-Term Memory Efficiency Under Forgetting

The goal of Experiment 2 is to determine whether the forgetting module can compress the knowledge graph without degrading its retrieval quality. A single persistent graph is built by ingesting the haystack sessions for all 500 questions in a sequence, simulating a long-running deployment in which a system accumulates memory across many independent interactions. Two variants are compared: a *no-forgetting* graph that retains every node, and a *forgetting* graph to which the forgetting module is applied once ingestion is complete. Both variants then answer all 500 benchmark questions, and their retrieval quality and storage footprints are compared.

Table 2: Experiment 2 results: storage and retrieval quality for the persistent graph with and without forgetting.

Variant	Nodes	Edges	Size	F1	Judge
No-forgetting	27,021	46,538	440.6 MB	0.292	0.300
Forgetting	24,368	43,978	398.6 MB	0.293	0.284
Change	-9.8%	-5.5%	-9.5%	+0.001	-0.016

5 Discussion and Limitations

The experiments reveal two complementary findings about graph-based long-term memory. First, representing conversational memory as a knowledge graph does not uniformly improve retrieval over a flat vector store. Second, once memory is represented as a persistent graph, selective forgetting can substantially reduce its size while largely preserving retrieval quality.

In Experiment 1, the baseline RAG system outperforms Graph RAG overall, achieving a token F1 of 0.468 compared with 0.417 and an LLM-judge accuracy of 0.536 compared with 0.454. However, performance varies across question types, indicating that the usefulness of graph structure depends on the type of information being retrieved.

Graph RAG achieves its only improvement on the LLM-judge metric for *temporal-reasoning* questions (0.293 vs. 0.278). This task is naturally aligned with a graph representation: events can be represented as typed nodes with temporal attributes and connected to the entities that participate in them, allowing retrieval to preserve relational and temporal structure. In contrast, flat chunk retrieval does not explicitly represent event participants, ordering, or temporal relationships.

The largest performance deficit occurs for *single-session (assistant)* questions (judge: 0.607 vs. 0.911). These questions often require recalling a specific recommendation or factual statement from a previous assistant response. The flat baseline can retrieve the original assistant turn verbatim, whereas graph extraction decomposes the turn into entities and relationships. In doing so, it may lose information about which item or statement was specifically emphasized. This highlights an important limitation of extraction-based memory representations: structured abstraction can improve relational organization while simultaneously discarding information required for precise or verbatim recall.

Graph RAG also underperforms on *knowledge-update* questions (F1: 0.456 vs. 0.511). Inspection of failures indicates that the current conflict-resolution policy can retain an earlier attribute value instead of replacing it with a more recent value when no explicit confidence score is available. A last-write-wins policy for appropriate factual and numeric attributes could therefore improve performance on knowledge-update tasks. Smaller deficits on *multi-session* and *single-session (user)* questions appear to arise from related extraction and retrieval effects. Although graph structure can support cross-session entity linking, imperfect extraction and entity merging introduce retrieval noise, while concise facts that are directly preserved in raw text may be abstracted during graph construction.

Experiment 2 examines a different property of the memory system: whether accumulated graph memory can be reduced without substantially degrading retrieval. Applying the forgetting mechanism removes 2,653 nodes (9.8%) and 2,560 edges (5.5%), reducing the graph size from 440.6 MB to 398.6 MB, a 9.5% reduction. Token-level F1 remains nearly unchanged, increasing from 0.292 to 0.293, while LLM-judge accuracy decreases from 0.300 to 0.284.

Absolute retrieval performance in Experiment 2 is lower than in Experiment 1 because all 500 haystacks are merged into a single persistent graph, introducing cross-conversation retrieval interference. The purpose of this experiment is therefore not to maximize retrieval accuracy, but to compare the same persistent-memory setting with and without forgetting.

The nodes removed by the forgetting mechanism fall below the importance threshold after the simulated conversation period and are characterized by low re-reference frequency, limited structural connectivity, and reduced recency. Their removal has little effect on token-level F1, suggesting that the importance function preferentially removes peripheral information that contributes relatively little to retrieval. The reduction in LLM-judge accuracy, however, indicates that some pruned information can still contribute to correct answers, highlighting a trade-off between memory efficiency and information retention.

Taken together, these results suggest that graph structure alone is not sufficient to improve long-term conversational memory. Its benefits are strongest when relationships and temporal structure are important, whereas flat text retrieval remains advantageous for precise or verbatim recall. At the same time, explicit retention mechanisms provide a practical way to control the growth of persistent memory. Effective long-term memory systems may therefore benefit from combining structured representations with stronger update policies, selective retention, and mechanisms that preserve access to information for which verbatim context remains important.

6 Conclusion

This study demonstrates that structuring conversational memory as a knowledge graph introduces both benefits and limitations. While relational representations can support reasoning over temporally and semantically connected information, they also incur information loss that negatively impacts tasks requiring precise or verbatim recall. These results indicate that improvements in memory systems cannot be achieved through representation alone. Instead, performance depends critically on how information is extracted, updated, and retained over time.

The proposed forgetting module contributes a retention mechanism whose cost we can bound: pruning the low-importance tail of a 27,021-node store removed 9.8% of nodes and 9.5% of bytes, and a paired bootstrap over 500 questions detects no significant change in any of the four metrics (Table 7). Overall, the findings suggest that effective long-term memory systems should combine structured representations with stronger update mechanisms and selective retention strategies, rather than relying on a single approach in isolation.

Author Contributions

Sourena Khanzadeh conceived the research idea and formulated the initial research direction. Theo Rusu was primarily responsible for the implementation and experimental execution. Manar Alalfi supervised the research and provided technical and academic guidance.

References

- Akari Asai, Zeqiu Wu, Yizhong Wang, Avi Sil, and Hannaneh Hajishirzi. Self-rag: Learning to retrieve, generate, and critique through self-reflection. In *International conference on learning representations*, volume 2024, pages 9112–9141, 2024.
- Jinheon Baek, Alham Fikri Aji, and Amir Saffari. Knowledge-augmented language model prompting for zero-shot knowledge graph question answering. In *Proceedings of the 1st Workshop on Natural Language Reasoning and Structured Explanations (NLRSE)*, pages 78–106, 2023.
- Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George Bm Van Den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, et al. Improving language models by retrieving from trillions of tokens. In *International conference on machine learning*, pages 2206–2240. PMLR, 2022.
- Lucas Bourtole, Varun Chandrasekaran, Christopher A. Choquette-Choo, Hengrui Jia, Adelin Travers, Baiwu Zhang, David Lie, and Nicolas Papernot. Machine unlearning. 2020. URL <https://arxiv.org/abs/1912.03817>.

- Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready ai agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.
- Hermann Ebbinghaus. *A contribution to experimental psychology*. 1913.
- Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitan, Robert Osazuwa Ness, and Jonathan Larson. From local to global: A graph rag approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024.
- Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, and Haofen Wang. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, 2023.
- Alex Graves, Greg Wayne, and Ivo Danihelka. Neural turing machines. *arXiv preprint arXiv:1410.5401*, 2014.
- Bernal J Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. Hipporag: Neurobiologically inspired long-term memory for large language models. volume 37, pages 59532–59569, 2024.
- Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Mingwei Chang. Retrieval augmented language model pre-training. In *International conference on machine learning*, pages 3929–3938. PMLR, 2020.
- Shaoxiong Ji, Shirui Pan, Erik Cambria, Pekka Marttinen, and Philip S Yu. A survey on knowledge graphs: Representation, acquisition, and applications. *IEEE transactions on neural networks and learning systems*, 33(2):494–514, 2021.
- Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 conference on empirical methods in natural language processing (EMNLP)*, pages 6769–6781, 2020.
- James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the national academy of sciences*, 114(13):3521–3526, 2017.
- Kuang-Huei Lee, Xinyun Chen, Hiroki Furuta, John Canny, and Ian Fischer. A human-inspired reading agent with gist memory of very long contexts. 2024.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. volume 33, pages 9459–9474, 2020.
- Lei Liu, Xiaoyan Yang, Yue Shen, Binbin Hu, Zhiqiang Zhang, Jinjie Gu, and Guannan Zhang. Think-in-memory: Recalling and post-thinking enable llms with long-term memory. *arXiv preprint arXiv:2311.08719*, 2023.
- Zihan Liu, Wei Ping, Rajarshi Roy, Peng Xu, Chankyu Lee, Mohammad Shoeybi, and Bryan Catanzaro. Chatqa: Surpassing gpt-4 on conversational qa and rag. *Advances in Neural Information Processing Systems*, 37:15416–15459, 2024.
- Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of llm agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 13851–13870, 2024.
- Michael McCloskey and Neal J Cohen. Catastrophic interference in connectionist networks: The sequential learning problem. 24:109–165, 1989.
- Ali Modarressi, Abdullatif Köksal, Ayyoob Imani, Mohsen Fayyaz, and Hinrich Schütze. Memllm: Finetuning llms to use an explicit read-write memory. *arXiv preprint arXiv:2404.11672*, 2024.

- Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G Patil, Ion Stoica, and Joseph E Gonzalez. Memgpt: Towards llms as operating systems. 2023.
- Shirui Pan, Linhao Luo, Yufei Wang, Chen Chen, Jiapu Wang, and Xindong Wu. Unifying large language models and knowledge graphs: A roadmap. *IEEE Transactions on Knowledge and Data Engineering*, 36(7):3580–3599, 2024.
- Joon Sung Park, Joseph O’Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th annual acm symposium on user interface software and technology*, pages 1–22, 2023.
- Ciyuan Peng, Feng Xia, Mehdi Naseriparsa, and Francesco Osborne. Knowledge graphs: Opportunities and challenges. 2023. URL <https://arxiv.org/abs/2303.13948>.
- Kurt Shuster, Spencer Poff, Moya Chen, Douwe Kiela, and Jason Weston. Retrieval augmentation reduces hallucination in conversation. In *Findings of the Association for Computational Linguistics: EMNLP 2021*, pages 3784–3803, 2021.
- Sainbayar Sukhbaatar, Jason Weston, Rob Fergus, et al. End-to-end memory networks. volume 28, 2015.
- Jiashuo Sun, Chengjin Xu, Lumingyuan Tang, Saizhuo Wang, Chen Lin, Yeyun Gong, Lionel Ni, Heung-Yeung Shum, and Jian Guo. Think-on-graph: Deep and responsible reasoning of large language model on knowledge graph. In *International Conference on Learning Representations*, volume 2024, pages 3868–3898, 2024.
- Bing Wang, Xinnian Liang, Jian Yang, Hui Huang, Zhenhe Wu, Shuangzhi Wu, Zejun Ma, and Zhoujun Li. Scm: Enhancing large language model with self-controlled memory framework. pages 188–203, 2025.
- Zhenyi Wang, Enneng Yang, Li Shen, and Heng Huang. A comprehensive survey of forgetting in deep learning beyond continual learning. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 47(3):1464–1483, 2024.
- Lei Wei, Xu Dong, Xiao Peng, Niantao Xie, and Bin Wang. Fademem: Biologically-inspired forgetting for efficient agent memory. pages 4011–4015, 2026.
- Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. Longmemeval: Benchmarking chat assistants on long-term interactive memory. 2024.
- Jing Xu, Arthur Szlam, and Jason Weston. Beyond goldfish memory: Long-term open-domain conversation. In *Proceedings of the 60th annual meeting of the association for computational linguistics (volume 1: long papers)*, pages 5180–5197, 2022.
- Zeyu Zhang, Quanyu Dai, Xiaohe Bo, Chen Ma, Rui Li, Xu Chen, Jieming Zhu, Zhenhua Dong, and Ji-Rong Wen. A survey on the memory mechanism of large language model-based agents. *ACM Transactions on Information Systems*, 43(6):1–47, 2025.
- Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. Memorybank: Enhancing large language models with long-term memory. In *Proceedings of the AAAI conference on artificial intelligence*, volume 38, pages 19724–19731, 2024.

A Appendix

A.1 Note to Reviewers on Experimental Scope and AI Use

Budget Constraints. This work was carried out under a fixed compute and API budget on the consumer-grade workstation described above. We state the resulting scope limits explicitly so that our claims are read at the right granularity. Every configuration we evaluate requires re-ingesting the haystack sessions for all 500 LongMemEval questions, which costs one extraction call per conversational turn, plus one generation and one judge call per question. A single evaluated configuration is therefore expensive, and the budget admitted a small number of complete runs rather than a sweep.

We chose to spend it on four full runs (Experiment 1 treatment and control, Experiment 2 treatment and control) at $n = 500$ with temperature = 0, and to report paired bootstrap intervals over those runs, rather than on a larger number of partially evaluated configurations.

Accordingly, our findings should be read as characterising this extraction-based graph memory pipeline at this model scale, not graph-structured memory in general. Given additional budget, our order of priority would be: a matched-compression control that prunes the same fraction of nodes at random, to isolate the contribution of the importance function; a second benchmark; and a stronger extraction model.

Use of Generative AI. We distinguish two uses of language models in this work. First, as components of the method itself: GPT-4o-mini performs knowledge-graph extraction and answer generation, and serves as the LLM judge, as described in Sections 3.3 and 4.1. Second, as authoring tools. In the latter role, we used gpt5.6 sol to draft prose in the appendix and to assist with literature search, and Opus 4.8 to assist with implementing the experimental pipeline. The main technical content, including the experimental design, the analyses, and the interpretation of results, is the authors’ own; model assistance on those sections was limited to grammar and formatting. All AI-assisted output was reviewed by the authors, all cited references were checked against their original sources, and the authors take full responsibility for the content of the paper.

A.2 Justification for parameter values chosen for the experiments and methodology.

Table 3 lists every free parameter of the system together with its setting and the basis on which it was chosen. We distinguish three cases: values selected empirically on a small probe set (E), values fixed *a priori* from a budget or cost constraint (B), and values fixed by convention or by symmetry with the baseline (C). We did not perform a full sweep over the retention parameters; the consequences of this are discussed at the end of this section.

Table 3: Parameters, settings, and basis for selection. E = empirical probe, B = budget or cost constraint, C = convention or symmetry with the baseline.

Parameter	Value	Basis
De-duplication cosine threshold τ_{dedup}	0.92	E
Retrieval cosine floor τ_{ret}	0.75	E
Retrieval roots (top- k)	5	C
Subgraph expansion depth	2 hops	B
Subgraph node cap	15	B
Forgetting interval	400 turns	B
Pruning threshold s_{min}	0.10	B
Recency half-life	90 days	C
Turn-decay half-life	1,000 turns	C
Scoring weights (w_r, w_f, w_c, w_t)	(0.35, 0.25, 0.20, 0.20)	C

De-duplication threshold. Selected by the probe procedure described in Appendix A.5. The value is deliberately conservative because the two error modes are not symmetric: a false merge collapses two distinct entities irreversibly and corrupts every edge incident to them, whereas a missed merge only leaves redundant nodes that later de-duplication passes or the retrieval stage can still surface. We therefore accepted a higher false-negative rate in exchange for a low false-merge rate.

Retrieval cosine floor. Descriptor embeddings produced by `nomic-embed-text` are anisotropic, so cosine similarity between unrelated short descriptors does not concentrate near zero. A floor of 0.75 admits paraphrases and partial mentions of the same entity while excluding nodes that are merely topically adjacent. This parameter is not load-bearing: because candidates are subsequently ranked and truncated to the top 5, the floor only affects queries for which fewer than five nodes clear it, in which case the system correctly retrieves a smaller context rather than padding it with unrelated nodes.

Number of retrieval roots. Set to 5 to match the number of chunks retrieved by the flat vector baseline (Section 4.2), so that the two systems are compared at an equal candidate-generation budget and any difference in performance is attributable to the representation rather than to the number of retrieval hits.

Expansion depth and node cap. One hop from a seed node returns only its immediate neighbours, which for most seeds is the set of attributes attached to a single entity and therefore adds little beyond the seed itself. Three or more hops expand super-linearly in a merged graph, and inspection showed that nodes at that distance are typically related to the seed through a hub entity rather than through any relation relevant to the query. Two hops is thus the smallest depth that supports the relational and multi-session cases the graph representation is intended to serve.

Forgetting interval and pruning threshold. Scoring is $O(N + M)$ in the size of the graph, so invoking it rarely amortizes its cost across many conversational turns; 400 turns is also long enough for the recency and turn-decay terms to separate nodes that are genuinely dormant from nodes that happen not to have been accessed recently. The threshold of 0.10 was set to be conservative, targeting only the tail of the score distribution rather than a fixed compression ratio. This choice determines the operating point reported in Experiment 2 and is the reason the reported reduction is approximately 10% rather than a larger figure.

Half-lives and scoring weights. The 90-day recency half-life is set relative to the temporal span of the LongMemEval haystacks, which cover roughly 33 months of real timestamps; a substantially shorter half-life would saturate the recency term for nearly all nodes, and a substantially longer one would flatten it. The 1,000-turn decay half-life plays the same role with respect to ingestion order. The weights were fixed *a priori* and were not tuned: they encode a prior, drawn from the memory-stream and forgetting-curve literature [Park et al., 2023, Zhong et al., 2024], that access-driven signals (recency, frequency) should dominate structural and age-based ones.

Limitation. The retention parameters (w_r, w_f, w_c, w_t), $s_{\min}$, and the forgetting interval were not swept, so Experiment 2 characterises a single point on the compression–quality trade-off rather than the curve. We also do not isolate the contribution of the individual scoring components; establishing that the four-term importance function outperforms a simpler retention rule at matched compression is left to future work.

Compute Environment. All experiments were conducted on a local workstation equipped with an AMD Radeon RX 7800 XT GPU, 16 GB of system RAM, and an Intel Core i5-9400F CPU. This configuration was used for local execution of the memory pipeline, graph operations, embedding-related workloads and we utilized OpenAI model, mainly (gpt4o-mini) for API calls.

A.3 Role-Aware Extraction Prompt

This role-aware variant explicitly distinguishes user and assistant turns and specifies which assistant-provided facts should be retained.

Role-Aware Extraction Prompt

Extract a knowledge graph from a conversation turn.
Return strict JSON only -- no markdown, no preamble.

Each turn is prefixed with its speaker role:
[Role: user] -- the human speaking directly
[Role: assistant] -- the AI assistant responding

ROLE HANDLING:
...

A.4 Sample Knowledge-Graph Extraction Prompt

The following example illustrates the structure of the extraction prompts used throughout our experiments.

Sample Knowledge-Graph Extraction Prompt

You are a knowledge graph extraction engine. Given a single conversational message, extract all relevant nodes, edges, and attributes and return the result as strict JSON.

NODES:
Represent discrete entities mentioned or implied in the message,

such as people, organizations, locations, skills, goals, events, preferences, and artifacts.

EDGES:

Represent directed relationships between nodes. Use precise, domain-relevant relationship labels whenever possible.

ATTRIBUTES:

Represent additional properties associated with a node or edge, including quantities, dates, durations, frequencies, or other qualifying information.

REQUIREMENTS:

1. Extract every explicitly stated fact.
2. Do not introduce information that is not supported by the input.
3. Use only the predefined node labels.
4. Return valid JSON only.
5. Do not include explanations, markdown, or additional commentary.

OUTPUT FORMAT:

```
{
  "nodes": {
    "<node_id>": {
      "label": "<node_label>",
      "title": "<short_title>",
      "content": "<description>",
      "attributes": {}
    }
  },
  "edges": {
    "<edge_id>": {
      "source": "<source_node>",
      "target": "<target_node>",
      "relationship": "<relationship>",
      "attributes": {}
    }
  }
}
```

USER MESSAGE:

<conversation turn>

A.5 Algorithms

Algorithm 1 summarizes the memory ingestion pipeline. Given a conversational turn, the system first extracts a structured set of nodes and edges and embeds each extracted node. Candidate nodes are matched against existing memory using exact title matching followed by semantic similarity. Matches above the deduplication threshold τ_{dedup} are mapped to existing nodes, while unmatched entities are assigned new identifiers. For standard operation, the same message is additionally converted into retrieval entities, which are used to identify and expand a relevant graph subgraph that is serialized as contextual memory. The extracted nodes and edges are then written to persistent memory, and an optional forgetting pass is triggered according to the configured memory-maintenance policy.

Deduplication Threshold Selection. To identify an appropriate deduplication threshold, we constructed a sequence of prompts containing repeated references to the same underlying entities while varying the wording and contextual phrasing across prompts. This allowed us to evaluate how consistently semantically equivalent entities were merged as the similarity threshold changed. We then selected the threshold that provided the best trade-off between correctly merging duplicate entities and avoiding incorrect merges between distinct entities.

A.6 Computational Complexity

Let N and M be the number of nodes and edges in the memory graph, d the embedding dimension, k the number of entities extracted from an incoming message, ℓ the number of relations extracted with them, and q the number of entities extracted from a query. Table 4 summarises the cost of each stage; we exclude the internal cost of the LLM and embedding calls, which depends on token lengths rather than on graph size.

Algorithm 1 Knowledge-Graph Ingestion Pipeline

Require: User message m , memory graph G **Ensure:** Updated graph G and retrieved context $\mathcal{C}$

```
1:  $E \leftarrow \text{EXTRACT}(m)$  ▷ Extract nodes and relations
2:  $Z \leftarrow \text{EMBED}(E.\text{nodes})$  ▷ Embed extracted entities
3: for all  $n \in E.\text{nodes}$  do
4:    $v \leftarrow \text{MATCH}(n, Z_n, G)$ 
5:   if  $v$  is sufficiently similar to  $n$  then
6:      $\text{MERGE}(n, v, G)$ 
7:   else
8:      $\text{ADDNODE}(n, G)$ 
9:   end if
10: end for
11:  $\text{ADDERELATIONS}(E.\text{edges}, G)$ 
12:  $Q \leftarrow \text{EXTRACTQUERYENTITIES}(m)$ 
13:  $R \leftarrow \text{RETRIEVERELEVANTNODES}(Q, G)$ 
14:  $S \leftarrow \text{EXPANDSUBGRAPH}(R, G)$ 
15:  $\mathcal{C} \leftarrow \text{SERIALIZE}(S)$ 
16: if  $\text{FORGETTINGTRIGGERED}(G)$  then
17:    $G \leftarrow \text{FORGET}(G)$ 
18: end if
19: return  $(\mathcal{C}, G)$ 
```

Table 4: Per-stage complexity under exhaustive vector search. T is the forgetting interval (400 turns), and V_S, E_S are the nodes and edges of the retrieved subgraph.

Stage	Cost	Note
Embed extracted entities	$O(kd)$	
Title de-duplication	$O(1)$ per entity	hash index
Embedding de-duplication	$O(kNd)$	linear scan
Graph write	$O(k + \ell)$	indexed updates
Candidate retrieval	$O(qNd)$	linear scan
Subgraph traversal	$O(N + M + V_S + E_S)$	adjacency build
Forgetting	$O(N + M)$ every T turns	amortized $O(\frac{N+M}{T})$

Two exhaustive vector scans dominate, one at de-duplication and one at retrieval, giving a worst-case per-turn cost of

$$O((k + q)Nd + N + M) = O(Nd + M),$$

since k, q , and the subgraph size are bounded by construction (the traversal is capped at 15 nodes). The $N + M$ term arises only because the adjacency representation is rebuilt at query time and would vanish if it were persisted alongside the graph. Space complexity is $O(Nd + N + M)$, where Nd accounts for stored embeddings and $N + M$ for graph structure and metadata.

The linear-scan terms are therefore the only components that grow with memory size, and both are incidental to the design: replacing the exhaustive search with an approximate nearest-neighbour index would reduce the $O(Nd)$ factor substantially, leaving the forgetting module as the mechanism that bounds N itself.

A.7 Statistical Significance of Experimental Results

We report the statistical significance of the results underlying our main claim, namely that the forgetting module handles long-term memory growth efficiently: it substantially reduces graph storage (Table 2) without a statistically significant loss in answer quality. All intervals below are computed post-hoc over the per-question results produced by our benchmark runs (Section 4); no additional model calls were made to compute them.

Setup. The factor of variability captured by every interval below is *which questions were sampled from the fixed LongMemEval evaluation set*, i.e. we resample over questions; answer generation

used temperature = 0, so there is no additional decoding-stochasticity component to capture. Rows corresponding to a crashed or errored run were dropped before aggregation. For each condition and metric we report the mean together with the standard error of the mean (SEM, a $1\text{-}\sigma$ interval, stated explicitly as such) and a 95% confidence interval obtained from a nonparametric bootstrap (10,000 resamples with replacement over questions, percentile method), which makes no Normality assumption on the underlying metric distribution. For the binary judge-correctness metric, whose sampling distribution is a proportion bounded in $[0, 1]$, we additionally report the 95% Wilson score interval, which by construction cannot extend outside $[0, 1]$; we prefer it over a symmetric interval for this metric to avoid the risk of implying out-of-range values.

Per-Condition Results. Table 5 reports the mean $\pm$ SEM and the 95% bootstrap confidence interval for each of the four experimental conditions.

Table 5: Mean $\pm$ SEM (1σ) and 95% bootstrap CI for each condition, $n = 500$ questions per condition.

Condition	F1	Precision	Recall	Judge acc.
Baseline RAG (per-question, Experiment 1 control)	0.468 ± 0.019	0.532 ± 0.019	0.486 ± 0.020	0.536 ± 0.022
Graph RAG (per-question, Experiment 1 treatment)	0.417 ± 0.019	0.505 ± 0.020	0.412 ± 0.019	0.454 ± 0.022
Persistent graph, no forgetting (Experiment 2 control)	0.292 ± 0.017	0.363 ± 0.018	0.295 ± 0.018	0.300 ± 0.021
Persistent graph, with forgetting (Experiment 2 treatment)	0.293 ± 0.017	0.359 ± 0.018	0.294 ± 0.018	0.284 ± 0.020

Table 6: 95% bootstrap confidence intervals corresponding to Table 5. For judge accuracy, the 95% Wilson score interval is shown alongside the bootstrap CI.

Condition	F1	Precision	Recall	Judge acc.
Baseline RAG (per-question, Experiment 1 control)	[0.432, 0.505]	[0.496, 0.571]	[0.447, 0.525]	[0.492, 0.580] (Wilson: [0.492, 0.579])
Graph RAG (per-question, Experiment 1 treatment)	[0.381, 0.454]	[0.467, 0.544]	[0.375, 0.451]	[0.410, 0.496] (Wilson: [0.411, 0.498])
Persistent graph, no forgetting (Experiment 2 control)	[0.260, 0.325]	[0.328, 0.399]	[0.261, 0.330]	[0.260, 0.342] (Wilson: [0.262, 0.342])
Persistent graph, with forgetting (Experiment 2 treatment)	[0.260, 0.326]	[0.323, 0.395]	[0.259, 0.329]	[0.244, 0.324] (Wilson: [0.246, 0.325])

Paired Comparisons. To assess whether the differences between paired conditions are statistically significant, we compute a paired nonparametric bootstrap over the per-question difference in each metric, matched by question ID between the two conditions being compared (10,000 resamples). We report the mean difference, its 95% bootstrap CI, and a two-sided bootstrap p -value (twice the smaller tail fraction of resampled differences crossing zero, capped at 1). A comparison is marked significant at the $\alpha = 0.05$ level when the 95% CI on the difference excludes zero. Results are shown in Table 7.

Table 7: Paired bootstrap comparisons between conditions. A positive mean difference favors the first-named condition; * denotes significance at $\alpha = 0.05$.

Comparison	F1	Precision	Recall	Judge acc.
Experiment 1: Graph RAG vs Baseline RAG	$-0.050^* [-0.085, -0.016]$	$-0.028 [-0.067, +0.011]$	$-0.073^* [-0.107, -0.039]$	$-0.082^* [-0.128, -0.034]$
Experiment 2: Forgetting vs No forgetting	$+0.001 [-0.015, +0.016]$	$-0.004 [-0.023, +0.014]$	$-0.001 [-0.017, +0.016]$	$-0.016 [-0.038, +0.006]$

Interpretation. None of the four metrics show a significant difference between the forgetting and no-forgetting conditions (Experiment 2), which is the key evidence for the no-quality-loss half of our main claim. The Graph RAG vs. Baseline RAG comparison (Experiment 1) is included for completeness but is not load-bearing for our main claim.